Что мы сделали и зачем

Объяснение для новичка: задание про рекомендательные системы, данные Yambda-50M, пайплайн с градиентным бустингом, результаты на текущий момент

Коротко. Мы собрали «противника»: обычную модель рекомендаций с признаками, придуманными вручную. Дальше задача задания, это построить автоматический способ придумывать признаки так, чтобы он обыграл ручной вариант. К моменту этого отчёта готова вся подготовка и ручная модель. Автоматический метод ещё не начат.

1. О чём вообще задание

Когда приложение предлагает вам трек, видео или товар, за этим стоит модель машинного обучения. Чаще всего это **градиентный бустинг**: алгоритм, который по таблице чисел предсказывает, понравится ли вам предмет. Числа в таблице называются **признаками** (или «фичами»). Например: «сколько раз вы слушали этого исполнителя за последнюю неделю».

Проблема в том, что такие признаки придумывают люди: «давайте посчитаем за 7 дней». Но почему 7, а не 10? Скорее всего, никто не проверял. Задание просит найти способ, который подбирает такие вещи автоматически и работает не хуже (а лучше всего, лучше) человека.

Что нужно сдать

- Рабочий пайплайн: данные, сплит по времени, бустинг, ручные признаки. Это «планка», которую надо побить.
- Идея автоматизации признаков и эксперименты, которые доказывают, что она работает.
- Отчёт (репозиторий на GitHub или PDF) с идеями, методом и цифрами.

2. Словарик терминов

Эти слова будут встречаться дальше. Вернитесь сюда, если что-то непонятно.

Термин	Что это простыми словами
Признак (фича)	Число, которое описывает ситуацию. Например, «сколько раз юзер лайкал треки этого артиста».
Label (метка)	Правильный ответ, который модель учится угадывать. У нас: 1, если юзер лайкнул трек, 0 если нет.
Градиентный бустинг	Алгоритм, который строит много маленьких деревьев решений, каждое исправляет ошибки предыдущих. Мы используем библиотеку LightGBM.
Окно (7 дней, 30 дней)	Период в прошлом, по которому считается признак. «Лайки за 7 дней» это лайки за последнюю неделю до момента прогноза.
Срез (cutoff)	Момент времени, «сегодня» для модели. Всё, что было до него, она знает. Всё, что после, нет.
Утечка данных (leak)	Когда в признаки случайно попадает информация из будущего. Модель тогда показывает отличные цифры на тесте и бесполезна в жизни. Самая частая ошибка в таких задачах.
Train / val / test	Три куска данных по времени: на train модель учится, на val мы подбираем настройки, на test один раз проверяем итог. Идут строго друг за другом (прошлое, потом будущее).
AUC	Насколько хорошо модель отличает «лайкнет» от «не лайкнет». 0,5 это как подбросить монетку, 1,0 это идеально.

Термин	Что это простыми словами
NDCG@10, Recall@10	Метрики ранжирования. Для каждого юзера модель выстраивает кандидатов по убыванию вероятности. Смотрим, попали ли лайкнутые треки в первую десятку (Recall@10) и насколько высоко (NDCG@10). Больше = лучше.
Бейзлайн	Простейший способ для сравнения (например, случайный порядок). Если ваша модель не лучше бейзлайна, она не нужна.

3. Данные: Yambda-50M

Мы взяли открытый датасет Яндекса: прослушивания и лайки в сервисе Яндекс.Музыка. Скачали его с Hugging Face через библиотеку `datasets`.

Файл	Что внутри	Размер
listens	Прослушивания: юзер, трек, время, органика или рекомендация, доля прослушанного, длина трека	46,5 млн строк
likes	Лайки: юзер, трек, время, органика или рекомендация	881 тыс. строк
artist_item_mapping	Какому артисту принадлежит каждый трек	9,27 млн строк
album_item_mapping	В каком альбоме каждый трек	9,65 млн строк

Сэмплирование

46 млн строк тяжело для ноутбука, поэтому мы случайно выбрали **2000 пользователей** и взяли **всю** их историю (а не случайные строки, иначе у каждого юзера получилась бы рваная история). Получилось **10,15 млн прослушиваний и 173 тыс. лайков**.

Что проверили	Результат	Вывод
Покрытие артистов маппингом	99,985%	почти у каждого трека известен артист
Покрытие альбомов маппингом	99,997%	почти у каждого трека известен альбом

4. Что мы сделали по шагам

Шаг 1. Превратили данные в задачу «угадай лайк»

Каждая строка в таблице это одно **прослушивание**. Метка: 1, если юзер лайкнул этот трек в течение часа после начала прослушивания, иначе 0. Доля единиц около 0,77%, то есть лайк это редкое событие. Для бустинга такого достаточно (около 78 тыс. «положительных» примеров).

Шаг 2. Разделили данные по времени

Последние две недели ушли в test, две недели перед ними в val, всё остальное в train. Именно по времени, а не случайно: в жизни мы всегда предсказываем будущее по прошлому. Случайное перемешивание дало бы утечку из будущего.

Выборка	Период	Строк в сырой таблице
train	всё до val	8,88 млн
val	14 дней перед test	631 тыс.
test	последние 14 дней	646 тыс.

Шаг 3. Научились считать признаки без утечки

Идея «срез + окно»: выбираем момент (срез). Признаки считаем **только** по событиям до среза. Кандидаты (какие треки оцениваем) берём из следующих 14 дней после среза. В код встроена проверка (`assert`), которая падает, если история залезла за срез.

Выборка	Признаки считаем по истории до…	Кандидаты берём из…
val	начала val	14 дней val
test	начала test	14 дней test
train	4 разных срезов, по одному на неделю назад от val	по 7 дней после каждого среза

Какие признаки посчитали (ручные, «планка»)

Группа	Примеры	Зачем
Юзер (u_)	сколько прослушиваний и лайков за 1, 7, 30 дней и за всё время, конверсия лайки/прослушивания, средняя доля дослушивания	активность и вкус человека
Трек (i_)	то же самое, но по треку: популярность, конверсия, как часто дослушивают	насколько трек «заходит» людям
Юзер × трек (ui_)	сколько раз юзер слушал и лайкал именно этот трек	самые сильные признаки: прямая связь пары
Юзер × артист (uar_), юзер × альбом (ual_)	сколько раз юзер слушал и лайкал треки этого артиста и альбома, во всех окнах	косвенная связь: любит артиста, значит, вероятно, лайкнет и его новый трек

> Всего получилось **70 колонок** (с учётом идентификаторов и метки). Окна 1, 7, 30 дней и «всё время» заданы одним списком `WINDOWS`. Это важно: именно этот список мы потом будем подбирать автоматически.

Шаг 4. Бейзлайны и первый бустинг

Сначала проверили, что метрики вообще работают: случайный порядок даёт AUC около 0,5 (как и должно быть). Порядок «по популярности трека» тоже около 0,5. Это ожидаемо: кандидаты это треки, которые юзер уже слушал, и общая популярность почти не помогает угадать личный лайк. Весь сигнал приходит из признаков «юзер × трек».

5. Проблема, которую мы нашли по дороге

Первая версия бустинга показала AUC 0,87, и это слишком хорошо для правды. Мы заподозрили артефакт в определении метки: если юзер лайкнул трек *раньше*, чем слушал его снова, метка всегда 0, и признак «юзер уже лайкал этот трек» превращается в подсказку «ответ точно 0». Модель учится не вкусу, а этому правилу.

Что сделали: убрали из кандидатов все пары, где лайк уже был до среза (они не могут стать «новым» лайком). Это честнее: модель теперь предсказывает именно **новые** лайки. После фильтра выборки уменьшились (val: 631 тыс. → 528 тыс. строк, test: 646 тыс. → 544 тыс.).

Честная оговорка. Мы предложили проверочную строчку, которая подтвердила бы гипотезу, но её результат в переписке не появился. Так что причина высокого AUC — наиболее вероятная, а не доказанная. Фильтр в любом случае логичен, и все дальнейшие цифры считаются уже после него.

6. Результаты на текущий момент

Метрики, которые мы реально получили

Модель	Выборка	AUC	NDCG@10	Recall@10
random (до фильтра)	val	0,503	0,043	0,053
random (до фильтра)	test	0,496	0,044	0,055
top-popular (до фильтра)	val	0,504	0,031	0,046
top-popular (до фильтра)	test	0,494	0,032	0,042
boosting minimal (до фильтра, с артефактом)	val	0,871	0,071	0,092
boosting minimal (до фильтра, с артефактом)	test	0,877	0,080	0,106
boosting full, 4 среза (после фильтра)	val	0,854	0,072	0,100
boosting full, 4 среза (после фильтра)	test	0,860	0,087	0,119

Строки на оранжевом фоне посчитаны на данных с артефактом метки, поэтому **сравнивать их напрямую со строками после фильтра нельзя**. Бейзлайны (random, top-popular) тоже ещё надо пересчитать на отфильтрованных данных. Это ближайший шаг.

Как это читать

- **Модель с признаками заметно лучше случайной:** NDCG@10 на тесте 0,087 против 0,044, Recall@10 0,119 против 0,055 (чуть больше чем в два раза). Но формально сравнивать надо с бейзлайнами, пересчитанными после фильтра.
- **AUC 0,85 не равно «модель почти идеальна».** AUC считается по всем юзерам сразу, а юзеры сильно различаются по доле лайков. Поэтому честнее смотреть NDCG и Recall, которые считаются внутри каждого юзера.
- **Важные признаки:** сверху в рейтинге средняя доля дослушивания (юзера и трека) и конверсия юзера за 1, 7 и 30 дней. Значит, окна реально влияют на качество, и есть что подбирать автоматически.
- **Выросла ли точность от 4 срезов вместо одного** (train вырос с 618 до 972 тыс. строк), пока не ясно. Прямого сравнения на отфильтрованных данных у нас нет.

7. Где мы на карте плана

Этап	Что это	Статус
0. Подготовка	датасет, окружение, подвыборка 2000 юзеров	Готово (не создан репозиторий на GitHub)
1. Сплит по времени и фичи без утечки	train/val/test, схема «срез + окно», assert на утечку	Готово
2. Бейзлайны	random, top-popular, бустинг на минимуме	Готово, но надо пересчитать после фильтра
3. Ручной бустинг	ручные признаки в 4 окнах, 4 среза для train; косвенные признаки по артистам и альбомам	Признаки собраны, ещё не обучено с ними. Не сделан Optuna
4. Автоматизация признаков	главная часть задания	Не начато
5. Эксперименты	таблицы, абляции, несколько сидов	Не начато
6. Отчёт	README или PDF с выводами	Не начато

Зелёный: готово. Жёлтый: частично. Серый: не начато.

8. Что делать дальше

Ближайший шаг: этап 3, сравнение трёх вариантов на одних и тех же данных. Запускается одной ячейкой из прошлого сообщения. Она посчитает:

Модель	Какие признаки	Что покажет
random, top-popular	—	нижняя планка на отфильтрованных данных
boosting minimal (4 cuts)	только «всё время» и is_organic	что даёт отсутствие временных окон
full manual (4 cuts)	все ручные признаки без артистов и альбомов	сила обычного подхода
full + artist/album (4 cuts)	плюс юзер × артист и юзер × альбом	насколько косвенные связи помогают. Это итоговая ручная планка

- **Optuna** (автоматический подбор настроек бустинга): чтобы «противник» был честно затюнен.
- **Этап 4, автоматизация.** Описываем признак шаблоном «сущность × группировка × окно × агрегат × событие» (например, «юзер × артист × 10 дней × сумма × лайки»), генерируем сотни комбинаций, оставляем те, что улучшают метрику на val. Так можно показать, что оптимальное окно не 7 дней, а, скажем, 10 или 21. Сверху можно добавить LLM-агента, который сам предлагает новые шаблоны.
- **Эксперименты и отчёт:** таблица «бейзлайны, ручной бустинг, автоматика», абляции, несколько случайных зёрен, примеры найденных признаков.

9. Типичные ловушки (чтобы не наступить)

- **Признаки текущей строки нельзя использовать, если они известны только после события.** Так, доля прослушанного трека (`played_ratio_pct`) в самой строке — утечка, а её среднее по прошлому — нормально.
- **Любое перемешивание данных ломает смысл.** Всегда делим по времени.
- **Слишком хорошая метрика — повод сомневаться.** AUC 0,87 как раз оказалась такой.
- **Фильтр кандидатов должен быть одинаковым в train, val и test**, иначе метрики не сопоставимы.
- **В отчёте сравниваем только честные цифры** одного и того же набора данных и сплита.

Файлы и код живут в ноутбуке `T-Lab.ipynb`. Рекомендуется сохранить таблицы признаков в parquet (`Xtr.to_parquet(...)`) и завести репозиторий, чтобы не терять работу при перезапуске ядра.